
Livret professionnel : Épreuve E1

Bloc de compétences 1 : Collecte, stockage et mise à disposition des données

Garbage in, garbage out

David LEGRAND

Juillet 2026

Certification RNCP37827, Développeur en intelligence artificielle (saveur agentique)

Projet QualiCheck, Simplon.co

Table des matières

0.1 Présentation du projet 1

0.2 Introduction 1

0.3 C1 : Automatiser l'extraction de données 2

0.4 C2 : Développer des requêtes SQL d'extraction 2

0.5 C3 : Développer des règles d'agrégation de données 2

0.6 C4 : Créer une base de données 2

0.7 C5 : Développer une API mettant à disposition le jeu de données 2

0.8 Synthèse des critères de performance 2

0.9 Annexe A : Personas QualiCheck 4

0.10 Annexe B : Pipeline d'ingestion, conception cible 5

0.11 Annexe C : Pipeline d'ingestion, déroulé réel 6

0.12 Annexe D : Modèle conceptuel de données (MCD) 7

0.13 Annexe E : Flux d'une requête sur l'API données 8

0.14 Annexe F : Flux d'exécution des migrations 9

0.1 Présentation du projet

QualiCheck est développé dans le cadre de la certification **RNCP37827**, Développeur en intelligence artificielle (saveur agentique). Il s'appuie sur le référentiel **Opquast** (entreprise française, 245 bonnes pratiques qualité web, actuellement en Version 5, 2025-2030), avec le soutien d'Élie Sloïm, fondateur d'Opquast. L'application assiste un auditeur qualité web dans l'analyse d'un site : un auditeur expert certifié Opquast qui veut aller vite sans rien oublier, ou un auditeur curieux non certifié qui pose une question en langage naturel. L'agent IA propose, l'auditeur valide toujours la décision finale.

US	Interface	Rôle
US0 : Ingestion	Script CLI	Import des 245 règles Opquast (API + scraping), enrichissement par agent IA, indexation vectorielle (pgvector)
US1 : Audit assisté	Interface web	Sélection de pages et de règles, génération de constats par l'agent IA, dialogue, validation humaine, rapport final
US2 : Question libre	Interface web	Soumission d'une URL ou d'une image, dialogue libre appuyé sur une recherche sémantique pure parmi les 245 règles

Ce document, comme les 4 autres livrables de la certification, porte sur ce même projet support. Le contexte ci-dessus n'est donc pas repris dans les livrets suivants.

0.2 Introduction

Le bloc 1 couvre la collecte, la préparation et la mise à disposition du référentiel Opquast (245 règles), au travers d'un pipeline d'ingestion en 7 étapes. Une étape du pipeline fait appel à un **agent LLM** pour enrichir chaque règle (stratégie d'analyse, guide d'analyse exploité ensuite par l'audit). Une **boucle de revue humaine** permet ensuite à un référent, interne ou expert externe désigné, de signaler les classifications à corriger, sans jamais redéclencher d'appel LLM en dehors d'un script d'administration dédié.

Ce document reflète l'état du projet à sa date de rédaction. Le développement se poursuit au fil de l'eau jusqu'au passage : **CHANGELOG.md** et le dépôt Git (branche **dev**) font foi de l'état réel au moment de la soutenance. Le

périmètre décrit ici s'enrichira mécaniquement une fois US1 et US2 construites, notamment le registre RGPD (C4), qui devra couvrir les traitements liés aux audits dès que ces fonctionnalités deviendront réelles.

0.3 C1 : Automatiser l'extraction de données

`app/ingestion/acquisition.py` combine deux sources : l'API REST publique d'Opquast (`fetch_api()`, qui fournit intitulé, objectifs, tags et phases) et un scraping ciblé (`scrape_rule()`) pour les deux seuls champs non exposés par l'API (`solution`, `controle`). Le tout est orchestré par `scripts/ingestion.py` : point de lancement, logging centralisé, arrêt fail-fast explicite sur toute règle incomplète, sauvegarde en base.

Sur le critère du « mix de sources » (service web, fichier de données, base de données, big data), seules 2 sources sur 5 sont couvertes réellement. Les 3 restantes ne correspondent à aucun besoin réel du projet (référentiel fixe de 245 lignes). Ce choix est assumé et documenté plutôt qu'ajouté artificiellement : `docs/jury/decisions/2026-07-28-couverture-sources-extraction-c1.md`.

0.4 C2 : Développer des requêtes SQL d'extraction

`app/ingestion/stockage.py::load_enriched_rules_from_db()` (hook `--resume`) réalise une sélection triée par `numero`, avec des jointures vers les tables d'association (`objectif_regle`, `phase_regle`, `regle_tag`). Les choix de sélection, de jointures et d'optimisation sont documentés dans `conception/2_ingestion/ingestion.md`.

L'optimisation est volontairement absente ici (motif naïf, une requête par collection et par règle) : il s'agit d'un usage administratif ponctuel sur 245 lignes, au coût négligeable. Le choix inverse est assumé dans `app/api_regles/regles.py`, qui regroupe les mêmes jointures en 3 requêtes au total, car ce contexte de service HTTP public rendrait le motif naïf coûteux (736 requêtes par appel). Même besoin, deux choix différents, chacun justifié par son contexte réel.

0.5 C3 : Développer des règles d'agrégation de données

`app/ingestion/aggregation.py::aggregate_rules()` valide plutôt qu'il ne nettoie. Des `field_validator` Pydantic (schéma `RuleAggregation`) rejettent toute règle dont un champ texte est vide ou dont `objectifs/phases` est une liste vide, sans correction silencieuse : un arrêt fail-fast déclenche une correction humaine. `tags` est volontairement le seul champ non validé, puisque 64 des 245 règles n'ont légitimement aucun tag. 13 tests unitaires couvrent ce module.

0.6 C4 : Créer une base de données

La modélisation suit Merise (`conception/1_BDD/bdd.md`, `conception/1_BDD/MLD_qualicheck.md`). PostgreSQL et pgvector ont été choisis et justifiés au regard des contraintes du projet. 12 migrations Alembic sont versionnées, appliquées et testées (`tests/test_migration.py`).

Conformité RGPD : `docs/rgpd/registre_traitements.md` documente les traitements réels (référentiel Opquast, hors champ RGPD ; jetons API nominatifs) ainsi que les traitements liés aux audits, couverts au fur et à mesure que ces fonctionnalités deviennent réelles. Le raisonnement complet est détaillé dans `docs/jury/decisions/2026-07-29-perimetre-registre-rgpd.md`.

0.7 C5 : Développer une API mettant à disposition le jeu de données

`app/api_regles/` (détail complet dans `conception/4_api_regles/api_regles.md`) expose `GET /regles` (filtrable), `GET /regles/{numero}`, `PATCH /regles/{numero}` (boucle de revue) et `GET /health`. La documentation OpenAPI est générée nativement (`/docs`, `/redoc`, `/openapi.json`). La lecture reste ouverte, sans authentification, car le référentiel Opquast est sous licence CC BY-SA 4.0 et chaque réponse `GET` porte l'attribution requise. L'écriture est protégée par un jeton Bearer, un par client nommé.

83 tests couvrent l'ensemble, et l'acceptance a été vérifiée en conditions réelles sur l'instance de staging (245 règles réellement servies).

0.8 Synthèse des critères de performance

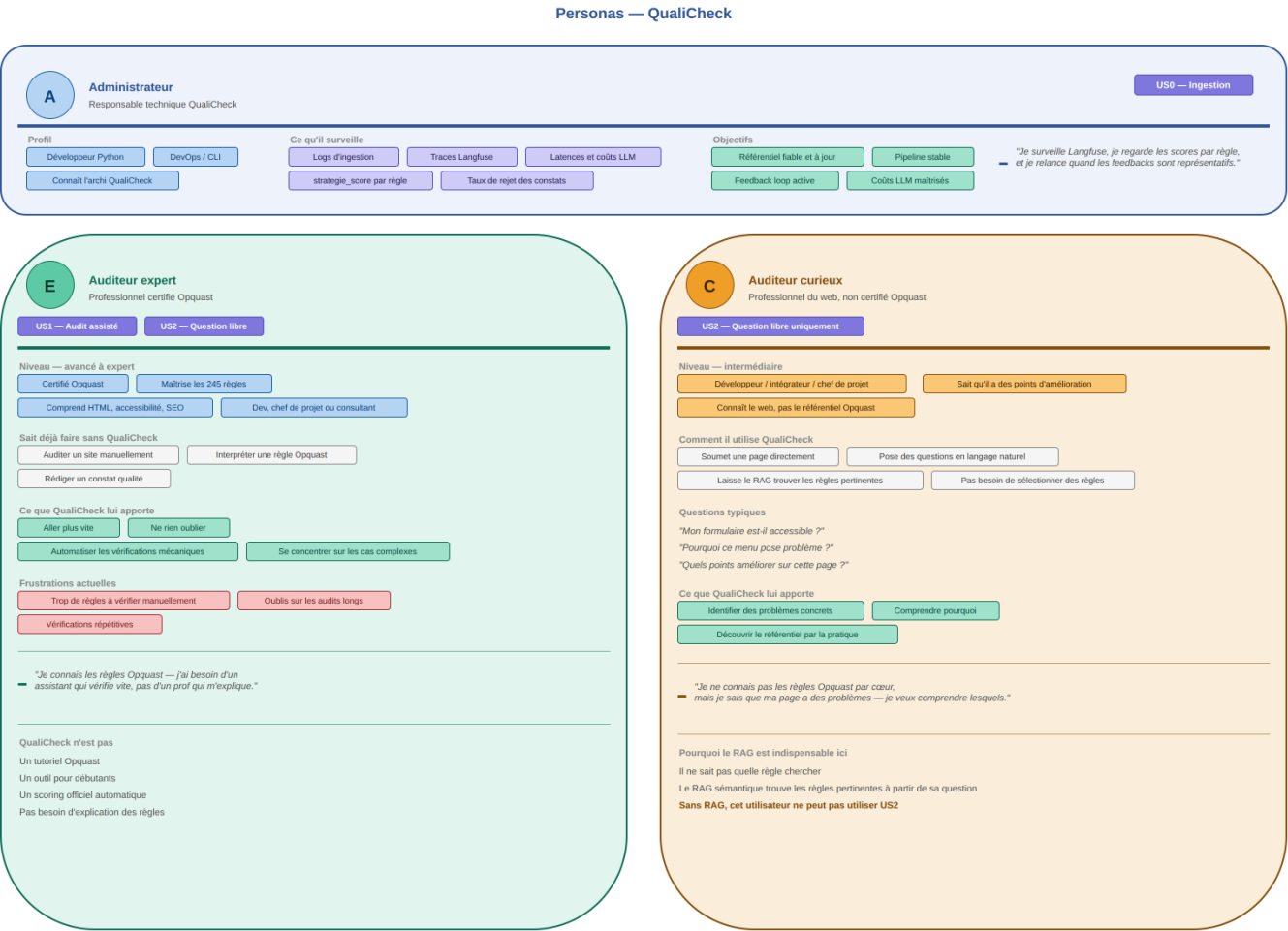
Compétence	Critères	Oui	Non	Non applicable
C1	4	3	0	1
C2	3	3	0	0
C3	3	3	0	0
C4	6	5	1	0
C5	4	4	0	0
Total	20	18	1	1

Le détail critère par critère, avec un commentaire pour chacun, se trouve dans `docs/jury/E1_bloc1_criteres_performance.xlsx`.

Les deux écarts ne sont pas des manques non traités, mais des choix documentés et assumés :

- Le seul critère non satisfait (registre RGPD « complet », C4) est différé à US1 (voir Introduction), le volet audit n'existant pas encore réellement.
- Le seul critère non applicable (mix de 5 sources d'extraction, C1) correspond à un besoin qui n'existe pas pour ce projet, dont le référentiel est fixe (245 lignes) : `docs/jury/decisions/2026-07-28-couverture-sources-extraction-c1.md`.

0.9 Annexe A : Personas QualiCheck



0.10 Annexe B : Pipeline d'ingestion, conception cible

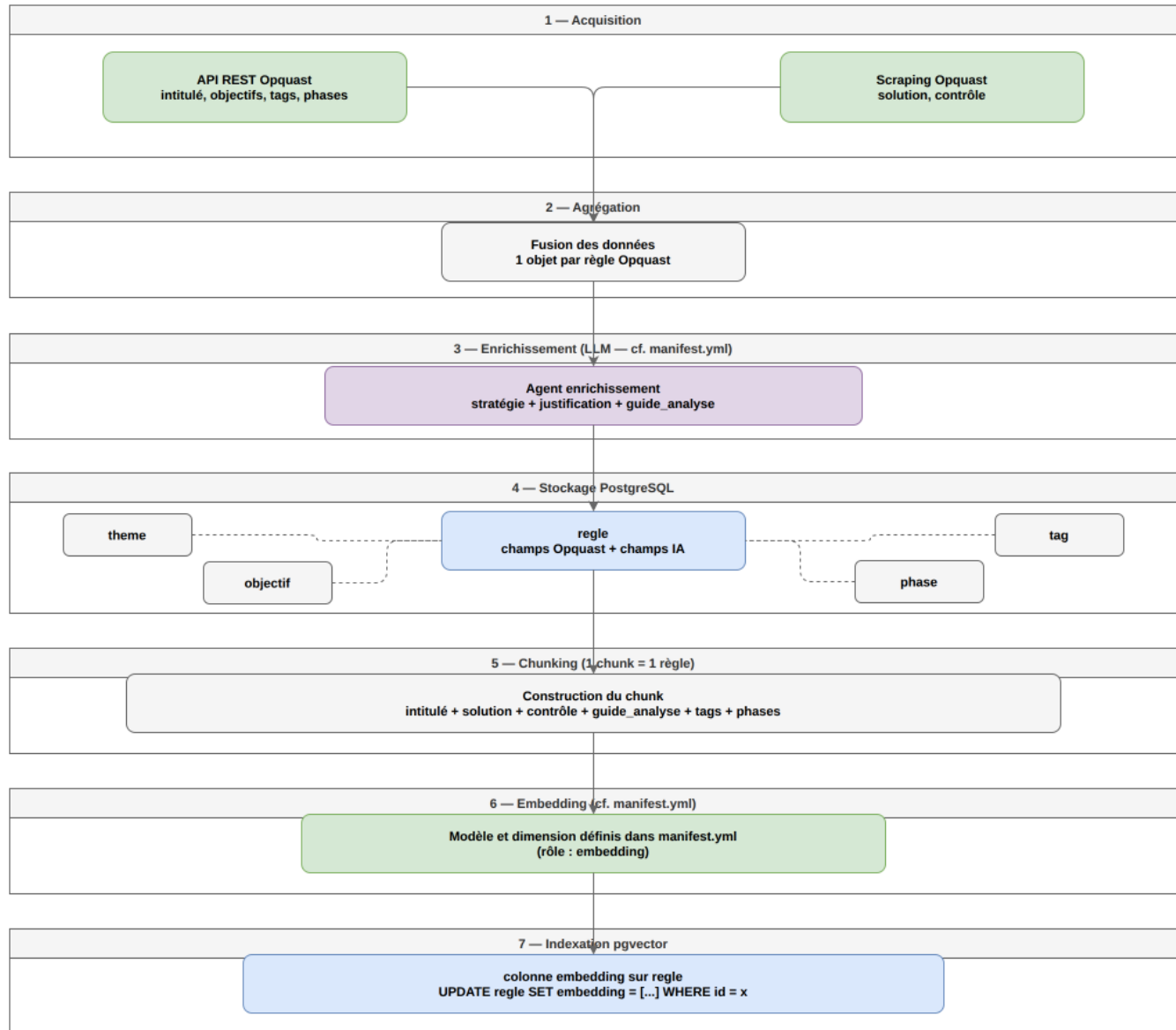


FIG. 2 : Pipeline d'ingestion en 7 étapes, conception cible

0.11 Annexe C : Pipeline d'ingestion, déroulé réel

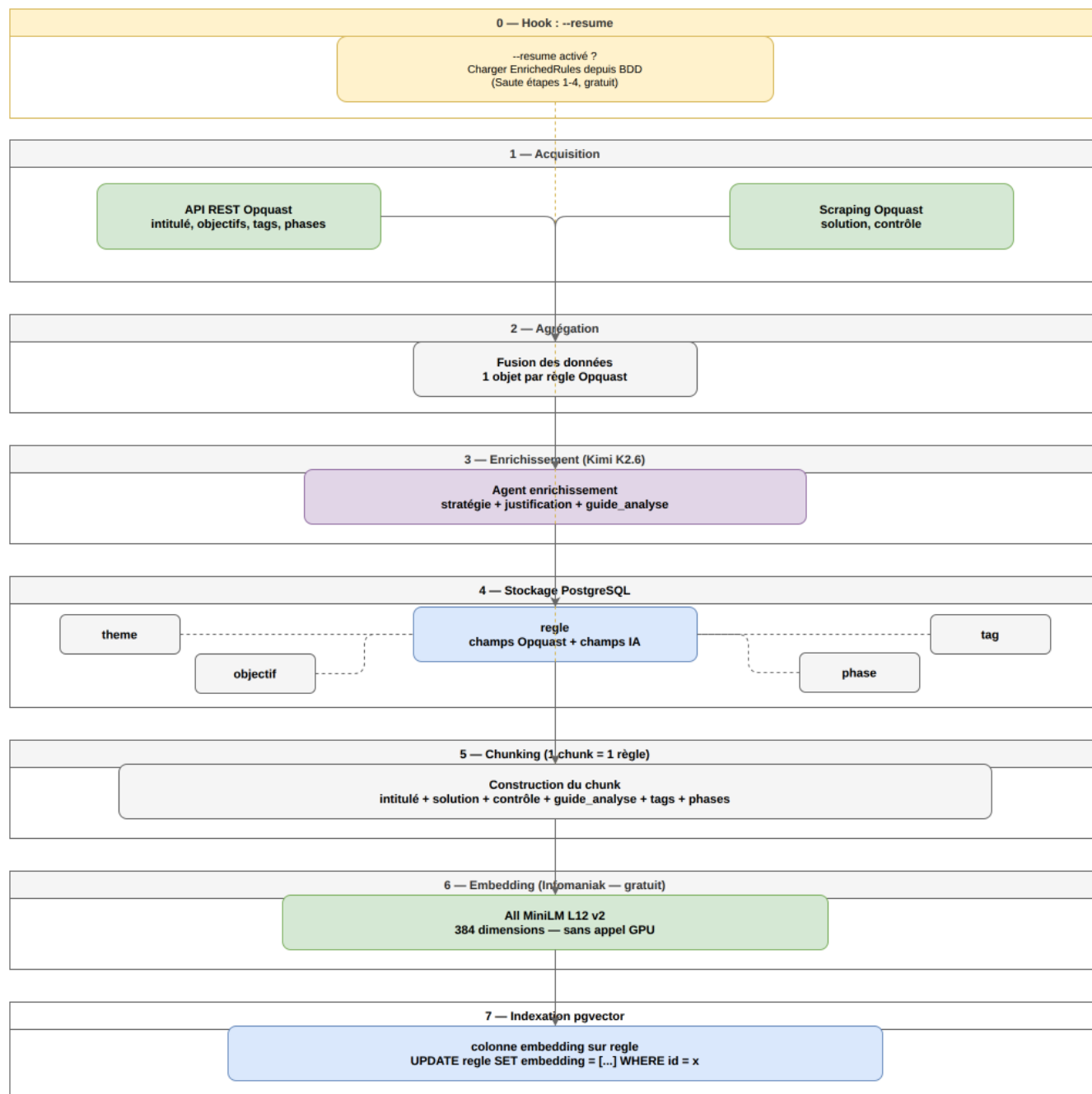


FIG. 3 : Pipeline d'ingestion réel, avec le hook --resume

0.12 Annexe D : Modèle conceptuel de données (MCD)

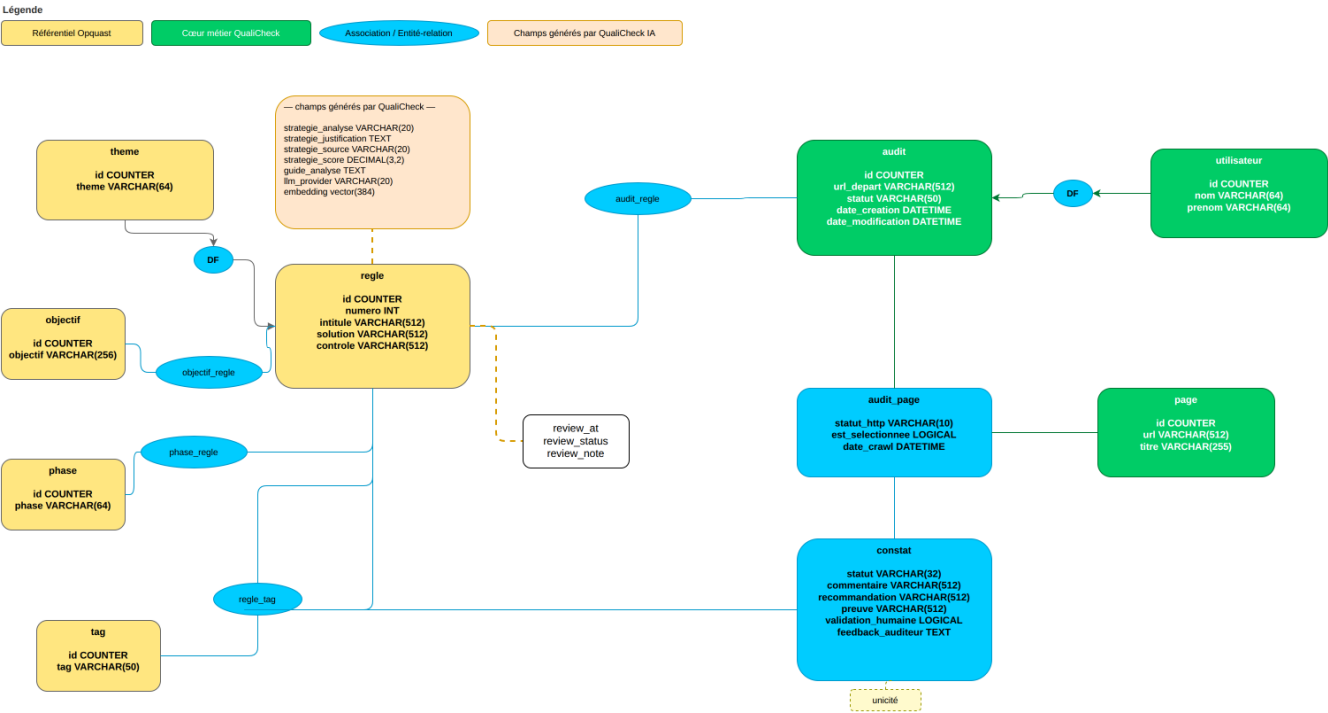


FIG. 4 : Modèle conceptuel de données QualiCheck

0.13 Annexe E : Flux d'une requête sur l'API données

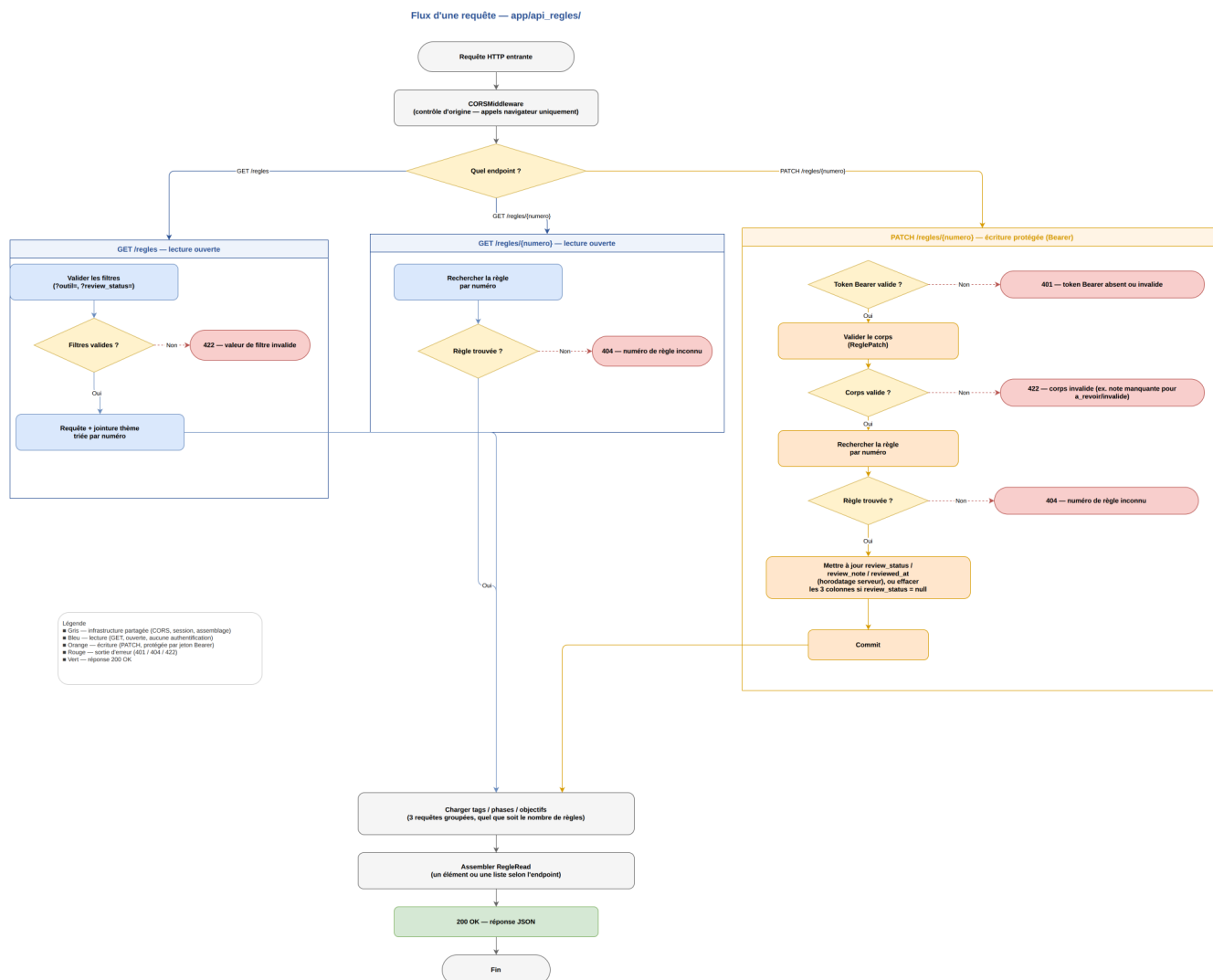


FIG. 5 : Flux d'une requête sur app/api_regles

0.14 Annexe F : Flux d'exécution des migrations

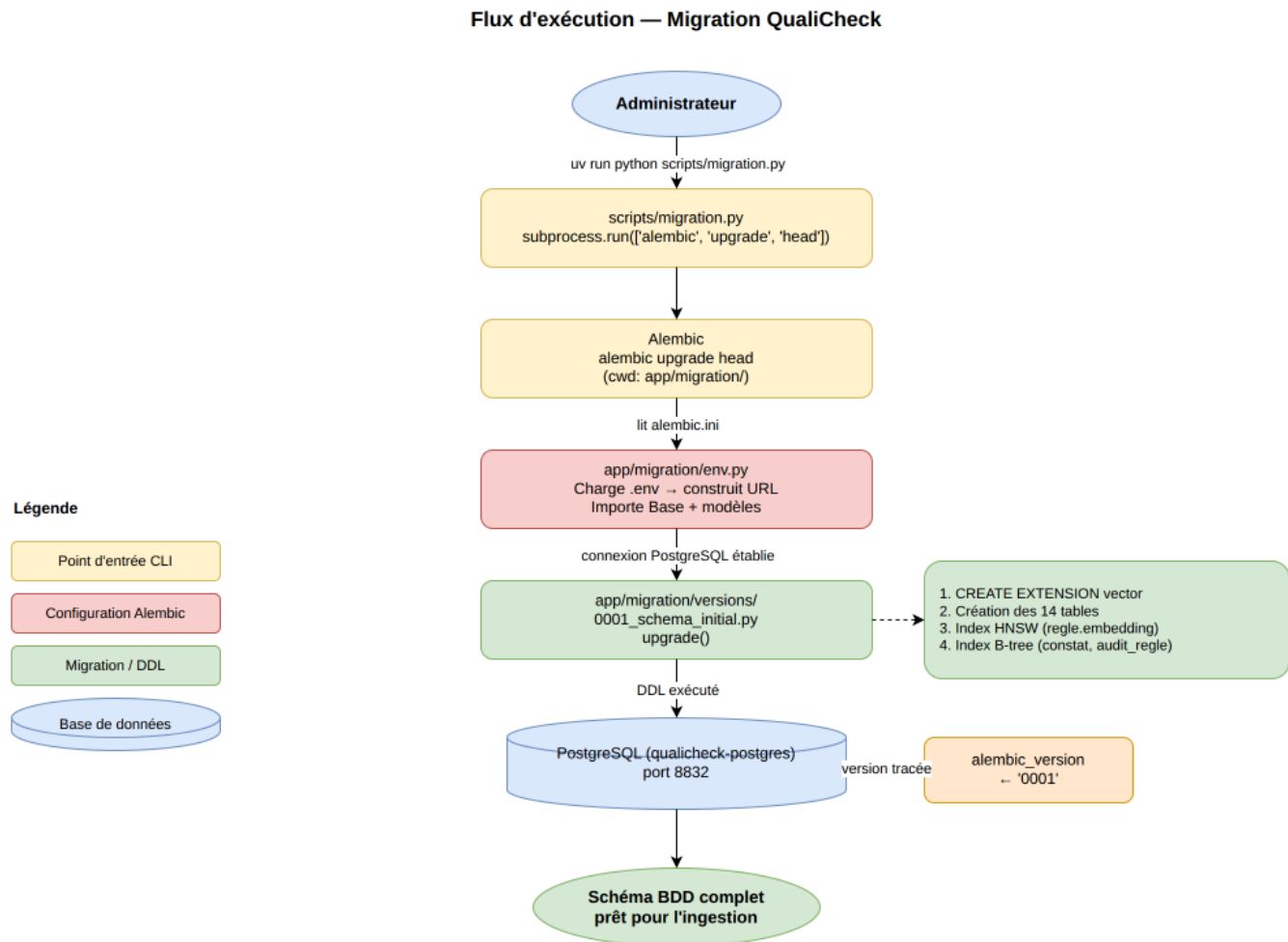


FIG. 6 : Flux d'exécution de la migration QualiCheck